Detecting Hallucinations in Large Language Model Outputs: A Feature-Based Classification Approach

Aaditya Dalal

University Institute of Engineering and Technology
Maharshi Dayanand University, Rohtak, Haryana
aadityadalal4@gmail.com

Code — https://github.com/dalalaadi30/Identifying-hallucinated-information-in-LLM-outputs

Abstract

Large Language Models (LLMs) are increasingly deployed in high-stakes domains such as healthcare, legal research, and education, yet they frequently generate factually incorrect or unsupported content, a phenomenon known as hallucination. Automated hallucination detection is a critical open problem in AI reliability and safety. This paper frames hallucination detection as a binary classification task: given a generated text response, a prediction is made as to whether it contains a hallucination. Eight interpretable NLP features are extracted from each response including response length, ROUGE-1, ROUGE-L, BLEU, semantic similarity, BERTScore F1, hedge count, and NLI entailment score and six classical machine learning classifiers are trained on the HaluEval summarization benchmark (Zhang et al. 2023). A central contribution of this work is the identification and documentation of a structural length bias in the HaluEval QA subset, where response length alone achieves an F1 score of 0.94, rendering the classification task trivially easy. This is corrected by switching to the summarization subset, where both correct and hallucinated summaries are comparable in length, requiring genuine semantic discrimination. On this more rigorous benchmark, the best model Gradient Boosting achieves an F1 of 0.728 and AUC of 0.801 on the held-out test set, comparable to GPT-4's reported $\sim$75% accuracy on the same benchmark, while requiring no GPU and no proprietary API at inference time. Domain shift is further identified as a key challenge when applying the detector to LLM responses on TruthfulQA (Lin, Hilton, and Evans 2022), motivating the need for domain-specific hallucination detectors.

Introduction

Large Language Models have demonstrated remarkable capabilities across a wide range of natural language tasks, from question answering and summarization to code generation and dialogue. However, despite their fluency and apparent confidence, these models frequently generate content that is factually incorrect, unverifiable, or inconsistent with provided source material, a phenomenon widely referred to as *hallucination* (Zhang et al. 2023). This failure mode poses serious risks in high-stakes deployment contexts. In the legal domain, LLM-generated hallucinations have led to fabricated case citations being submitted to courts. In healthcare, incorrect medical information generated with high confidence can directly endanger patients. In education, students relying on LLM outputs for research may unknowingly propagate misinformation.

The challenge of hallucination detection is compounded by the nature of the phenomenon itself. Hallucinated content is often fluent, contextually plausible, and stylistically indistinguishable from factually correct output. Unlike grammatical errors or off-topic responses, hallucinations require semantic and factual grounding to detect. They can only be identified by comparing generated content against a reliable reference. This makes automated detection significantly harder than standard text quality metrics would suggest.

Existing approaches to hallucination detection fall into three broad categories. *LLM-as-a-Judge* methods use a second, more powerful language model to evaluate the output of a first, achieving high accuracy but at significant computational and financial cost. *Uncertainty-based methods*, such as SelfCheckGPT (Manakul, Liusie, and Gales 2023), detect hallucinations by sampling multiple responses and measuring consistency—requiring no external knowledge but producing lower F1 scores and multiple API calls per query. *Feature-based classification* approaches extract numerical signals from generated text and train supervised classifiers, offering a lightweight, interpretable, and cost-effective alternative that does not require access to model internals or proprietary APIs.

This paper pursues the third approach. Hallucination detection is framed as a binary classification task, and eight interpretable NLP features are extracted from generated summaries, including lexical overlap metrics (ROUGE, BLEU), embedding-based semantic similarity, BERTScore F1, and Natural Language Inference (NLI) entailment scores. Six classical machine learning classifiers are systematically trained and evaluated on the HaluEval benchmark, with stratified cross-validation employed for hyperparameter tuning and a held-out test set used for final evaluation.

A central and novel contribution of this work is the identification of a *structural length bias* in the HaluEval QA subset. Through ablation analysis, it is demonstrated that response length alone achieves an F1 score of 0.94 on the

Copyright © 2025, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

QA subset, nearly identical to the full feature set indicating that classifiers learn to exploit a dataset artifact rather than genuine semantic hallucination patterns. This investigation is documented transparently, and the summarization subset is adopted instead, where both correct and hallucinated summaries are comparable in length. On this more rigorous benchmark, the best model achieves F1 of 0.728 and AUC of 0.801, comparable to GPT-4's reported $\sim$75% accuracy on the same benchmark, while operating entirely on CPU with no proprietary API dependency at inference time.

The remainder of this paper is organized as follows. Section 2 provides technical background on the algorithms and metrics used. Section 3 discusses related work. Section 4 defines the problem and describes the datasets. Section 5 details the methodology. Section 6 presents empirical results and analysis. Section 7 discusses real-world impact. Section 8 concludes with lessons learned and future directions.

Background

This section provides technical background on the machine learning algorithms, NLP features, and evaluation metrics used in this work.

Classification Algorithms

Logistic Regression is a linear classifier that models the probability of a binary outcome using a sigmoid function applied to a weighted sum of input features. Despite its simplicity, logistic regression is a strong baseline for text classification tasks where features exhibit approximately linear relationships with the target variable. Regularization via L1 (Lasso) or L2 (Ridge) penalties controls model complexity and prevents overfitting to noisy features.

Support Vector Machines (SVM) find the maximum-margin hyperplane separating two classes in feature space (Cortes and Vapnik 1995). The kernel trick enables SVMs to operate in high-dimensional or non-linear feature spaces without explicitly computing the transformation. The radial basis function (RBF) and linear kernels are the most commonly used for text classification.

Decision Trees partition feature space through a sequence of binary splits, each chosen to maximize class purity according to a criterion such as Gini impurity or information gain. While highly interpretable, single decision trees are prone to overfitting and high variance, motivating the use of ensemble methods.

Random Forests address the limitations of single decision trees by training an ensemble of trees on bootstrap samples of the training data, with random feature subsets considered at each split (Breiman 2001). Final predictions are determined by majority vote, reducing variance and improving generalization.

Gradient Boosting (XGBoost) builds an ensemble of weak learners sequentially, with each tree trained to correct the residual errors of all previous trees (Chen and Guestrin 2016). XGBoost is a highly optimized implementation with built-in regularization, subsample randomization, and feature column subsampling, making it among the strongest classifiers for tabular feature sets.

K-Nearest Neighbors (KNN) classifies a new example by finding the k most similar training examples in feature space and taking a majority vote. Performance depends heavily on the choice of distance metric and the number of neighbors k.

NLP Features

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) (Lin 2004) measures n-gram overlap between a hypothesis and a reference text. ROUGE-1 computes unigram overlap, while ROUGE-L measures the longest common subsequence, capturing phrase-level alignment. Both are reported as F1 scores combining precision and recall.

BLEU (Bilingual Evaluation Understudy) (Papineni et al. 2002) measures n-gram precision of a hypothesis against one or more references, with a brevity penalty for short outputs. Smoothed sentence-level BLEU is used to handle cases where n-gram matches are sparse.

Semantic Similarity is computed as the cosine similarity between dense sentence embeddings produced by the `all-MiniLM-L6-v2` model (Reimers and Gurevych 2019). This measures the angular distance between two embedding vectors, with values of 1.0 indicating identical semantic direction.

BERTScore (Zhang et al. 2020) computes token-level semantic similarity between hypothesis and reference using contextual embeddings from a pretrained BERT model. For each token in the hypothesis, the most similar token in the reference is identified using cosine similarity, and an F1 score is computed over these soft token matches. Unlike ROUGE, BERTScore captures paraphrastic similarity and is sensitive to specific word choices.

NLI Entailment Score uses a Natural Language Inference model to classify the relationship between a premise and hypothesis as entailment, contradiction, or neutral. The `cross-encoder/nli-deberta-v3-small` model (He et al. 2021) is used, treating the source document as the premise and the generated summary as the hypothesis. The output is encoded as a signed confidence score: positive for entailment, negative for contradiction, and zero for neutral.

Hedge Count measures the number of uncertainty markers in a response, including phrases such as "might," "possibly," and "it seems." This signal proved weak in practice for the summarization task, as both hallucinated and correct summaries were stated with similar levels of confidence.

Evaluation Metrics

F1 Score is the harmonic mean of precision and recall, defined as $F1 = 2 \cdot \frac{P \cdot R}{P+R}$, and serves as the primary metric due to its balance between false positives and false negatives.

Precision measures the fraction of predicted hallucinations that are true hallucinations: $P = \frac{TP}{TP+FP}$.

Recall measures the fraction of actual hallucinations that are correctly identified: $R = \frac{TP}{TP+FN}$.

AUC-ROC (Area Under the Receiver Operating Characteristic Curve) measures discriminative ability across all

classification thresholds, complementing F1 by evaluating model performance independent of threshold choice.

Related Work

Hallucination Benchmarks

Zhang et al. introduced HaluEval, a large-scale hallucination evaluation benchmark containing 35,000 samples across question answering, dialogue, and summarization tasks (Zhang et al. 2023). Hallucinated responses were generated by prompting ChatGPT to produce plausible but factually incorrect content. Their evaluation demonstrated that even GPT-4 achieves only $\sim$75% accuracy at recognizing hallucinations on the summarization subset. This work builds directly on HaluEval, using the summarization subset as the primary training and evaluation benchmark, with results directly comparable to those reported by Zhang et al.

Lin, Hilton, and Evans introduced TruthfulQA, a benchmark of 817 adversarially designed questions targeting common human misconceptions (Lin, Hilton, and Evans 2022). A key finding is that larger models are not necessarily more truthful—GPT-3 achieved only $\sim$58% truthfulness. TruthfulQA is used here as a secondary dataset to generate LLM responses for cross-model analysis, though domain shift prevents direct application of the trained classifier to this data, a finding discussed as a key limitation.

Hallucination Detection Methods

Manakul, Liusie, and Gales proposed SelfCheckGPT, a zero-resource method that detects hallucinations by sampling multiple responses and measuring consistency (Manakul, Liusie, and Gales 2023). While training-free, it requires multiple API calls per query and achieves F1 scores of approximately 0.44 on adversarial QA benchmarks.

Min et al. introduced FActScoring, which decomposes LLM responses into atomic factual claims and verifies each against a reference knowledge base (Min et al. 2023). This approach achieves strong results but requires access to a comprehensive external knowledge base, limiting its scalability.

How This Work Differs

Unlike LLM-as-a-Judge approaches, the proposed method requires no proprietary API at inference time. Unlike SelfCheckGPT, only a single response per query is required, and interpretable feature-level explanations are produced. Unlike FActScoring, no external knowledge base is needed. The primary contribution relative to HaluEval (Zhang et al. 2023) is a systematic multi-classifier benchmarking study with explicit feature importance analysis, ablation experiments documenting the QA length bias, and a discussion of domain shift as a practical deployment challenge.

Problem Formulation and Dataset

Problem Definition

Hallucination detection is framed as a **binary classification** task. Given a generated text summary s and its corresponding source document d, the goal is to learn a function $f : \mathcal{X} \to \{0, 1\}$ where $\mathcal{X}$ is the feature space derived from (s, d), and the output label $y \in \{0, 1\}$ indicates whether s contains a hallucination ($y = 1$) or is factually consistent with d ($y = 0$).

Table 1: Length bias ablation on HaluEval QA subset.

Feature Set	F1 Score
Response length only	0.9407
All 8 features (QA subset)	0.9464
All 8 features without length	0.8400
All 8 features (summarization)	0.7376

Primary Dataset: HaluEval Summarization

The HaluEval benchmark (Zhang et al. 2023) provides 10,000 summarization samples derived from the CNN/Daily Mail corpus (Hermann et al. 2015). Each sample consists of a source news article, a correct summary, and a hallucinated summary generated by prompting ChatGPT to introduce subtle factual errors while preserving fluency.Binary hallucination labels are provided for all samples.

A total of 5,000 rows are sampled using stratified sampling, yielding a perfectly balanced dataset of 2,500 hallucinated and 2,500 non-hallucinated summaries. An 80/20 stratified train/test split produces a training set of 4,000 rows and a held-out test set of 1,000 rows.

Key dataset properties:

- **Class balance:** 50% hallucinated, 50% not hallucinated — no class imbalance handling required
- **Summary length:** Correct summaries average 50.3 words; hallucinated summaries average 70.4 words
- **Hallucination type:** Subtle factual errors embedded in otherwise fluent, plausible text
- **Missing values:** None — all feature columns are fully populated after computation

Dataset Investigation: Length Bias in HaluEval QA

Prior to selecting the summarization subset, the HaluEval QA subset was initially used. Inspection revealed a critical structural artifact: correct answers were predominantly short factual extractions (mean 2.3 words), while hallucinated answers were full fabricated sentences (mean 10.9 words). An ablation study was conducted to quantify this bias, with results shown in Table 1.

Response length alone achieves F1 of 0.94, nearly identical to the full feature set. Removing response length drops F1 by 10.6 points, confirming that classifiers exploit answer length rather than semantic hallucination patterns. Feature importance analysis (Section 6) further confirms this finding. The summarization subset is therefore adopted, reducing F1 to a realistic 0.74 and requiring genuine semantic discrimination.

Secondary Dataset: TruthfulQA

TruthfulQA (Lin, Hilton, and Evans 2022) contains 817 questions adversarially designed to elicit hallucinations

Table 2: Mean feature values by label on HaluEval summarization.

Feature	Correct	Hallucinated	Dir.
Response length	50.3	70.4	↑
Hedge count	0.152	0.166	≈
ROUGE-1	0.152	0.183	↑
ROUGE-L	0.111	0.119	↑
BLEU	0.002	0.003	↑
Semantic sim.	0.749	0.759	≈
BERTScore F1	0.783	0.779	↓
NLI score	+0.010	−0.039	↓

from LLMs across 38 categories. Three LLMs were queried on all 817 questions via free-tier APIs:

- **Mistral Small** — via Mistral AI API (free tier)
- **Llama 3.3 70B** — via Groq API (free tier)
- **Llama 3.1 8B** — via HuggingFace Inference Providers (SambaNova backend)

These models provide diversity across model families (Mistral vs. Meta Llama), model sizes (8B vs. 70B parameters), and serving infrastructure, while remaining within free API tier constraints. The same eight NLP features were computed on each LLM response against TruthfulQA reference answers, producing an 817×24 feature matrix (8 features $\times$ 3 models).

Feature Engineering

Table 2 summarizes the eight features computed per response with mean values by label on the HaluEval summarization subset. Notably, most features exhibit higher values for hallucinated summaries which is the opposite of intuition. This reflects that ChatGPT- generated hallucinations borrow vocabulary from the source document to sound plausible while distorting facts. Only BERTScore F1 and NLI score move in the expected direction.

Methodology

Pipeline Overview

The hallucination detection pipeline consists of four stages: (1) dataset loading and preprocessing, (2) feature engineering, (3) model training and hyperparameter tuning, and (4) evaluation. All experiments were conducted on CPU without GPU acceleration, using Python 3.10 with scikit-learn, XGBoost, HuggingFace Transformers, and sentence-transformers.

Preprocessing

Raw text responses were lowercased and whitespace-normalized before feature computation. Class balance was ensured by stratified sampling. All numerical features were standardized to zero mean and unit variance using `StandardScaler` fitted on the training set only, with the same scaler applied to the test set to prevent data leakage. Feature scaling was applied uniformly across all six classifiers via scikit-learn `Pipeline` objects.

Table 3: Best hyperparameters identified via tuning.

Model	Best Parameters
Logistic Regression	C=0.1, penalty=L2
SVM	C=10, kernel=linear
Decision Tree	criterion=entropy, max_depth=5
Random Forest	n_estimators=100, max_depth=10, max_features=log2
Gradient Boosting	n_estimators=100, max_depth=3, lr=0.1
KNN	n_neighbors=15, metric=euclidean

Classifier Selection and Justification

Six classifiers were selected to provide broad coverage of the hypothesis space. **Logistic Regression** serves as the primary linear baseline with directly interpretable coefficient weights. **SVM** extends this with kernel-based non-linear decision boundaries. **Decision Tree** provides a fully interpretable single-model baseline, enabling quantification of the benefit of ensembling. **Random Forest** reduces decision tree variance through bootstrap aggregation and provides native feature importance. **Gradient Boosting (XGBoost)** sequentially minimizes classification error with built-in regularization, making it robust on tabular feature sets. **KNN** provides a non-parametric distance-based reference with no distributional assumptions.

Hyperparameter Tuning

Tuning was performed using stratified 5-fold cross-validation on the training set, optimizing macro F1. `GridSearchCV` was used for models with small parameter grids (Logistic Regression, SVM, Decision Tree, KNN) and `RandomizedSearchCV` with 20 iterations for larger search spaces (Random Forest, Gradient Boosting). Table 3 summarizes the best parameters found.

Validation Strategy

Hyperparameter tuning was conducted exclusively on the training set via stratified 5-fold cross-validation. The held-out test set (1,000 rows) was used only once after all tuning decisions were finalized to report final performance metrics, ensuring an unbiased estimate of generalization performance.

Feature Importance Analysis

Feature importance is extracted from three models: native Gini impurity-based importance from Random Forest, gain-based importance from Gradient Boosting (XGBoost), and learned coefficient magnitudes from Logistic Regression. These three perspectives provide complementary views of feature utility across linear and non-linear models.

Table 4: Baseline results: untuned 5-fold CV on training set.

Model	F1	AUC	Accuracy
Gradient Boosting	0.7329	0.8086	0.7332
Logistic Regression	0.7381	0.8167	0.7448
Random Forest	0.7379	0.8127	0.7403
SVM	0.7331	0.8198	0.7430
KNN	0.7054	0.7649	0.7067
Decision Tree	0.6662	0.6662	0.6663

Empirical Results and Evaluation

Baseline Results

Table 4 reports 5-fold cross-validation results for all six classifiers in their untuned configurations on the training set. Gradient Boosting leads with F1 of 0.757, followed closely by Random Forest (0.738) and Logistic Regression (0.738). Decision Tree is the clear outlier, achieving AUC equal to its F1 indicating the absence of probabilistic calibration in untuned single trees.

Tuned Model Results

Table 5 reports performance of all tuned models on the held-out test set. Gradient Boosting achieves the highest F1 of 0.728, followed closely by Random Forest (0.726) and Logistic Regression (0.723). Notably, SVM achieves the highest AUC of 0.806 despite lower F1, indicating superior probabilistic ranking ability. The top five models are separated by only 0.007 F1 points, confirming that the feature set rather than model choice is the primary performance bottleneck.

Effect of Hyperparameter Tuning

Table 6 compares baseline and tuned F1 scores. Tuning provided the greatest benefit to models most prone to overfitting: Decision Tree improved by +0.024 F1 through depth constraint, and KNN improved by +0.016 through neighbor count and distance metric selection. Ensemble methods (Random Forest, Gradient Boosting) showed marginal improvement, as their ensemble mechanisms already provide implicit regularization.

The negative ΔF1 values for ensemble methods and Logistic Regression reflect the generalization gap between cross-validation performance (training distribution) and held-out test performance (unseen data). This gap of approximately 0.01–0.015 F1 is expected and indicates well-calibrated models without excessive overfitting.

Feature Importance Analysis

Figure 1 shows feature importance plots for Random Forest, Gradient Boosting, and Logistic Regression coefficients. Across all three models, `response_len` is the most important feature, accounting for approximately 45% of Random Forest importance and 82% of Gradient Boosting importance. This finding is consistent with the length bias ablation in Table 1 and reflects the residual length difference between correct (50.3 words) and hallucinated (70.4 words) summaries even in the summarization subset.

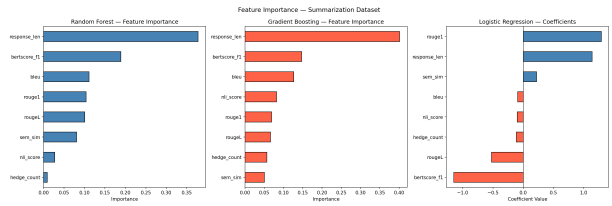


Figure 1: Feature importance for Random Forest (Gini), Gradient Boosting (gain), and Logistic Regression (coefficients).

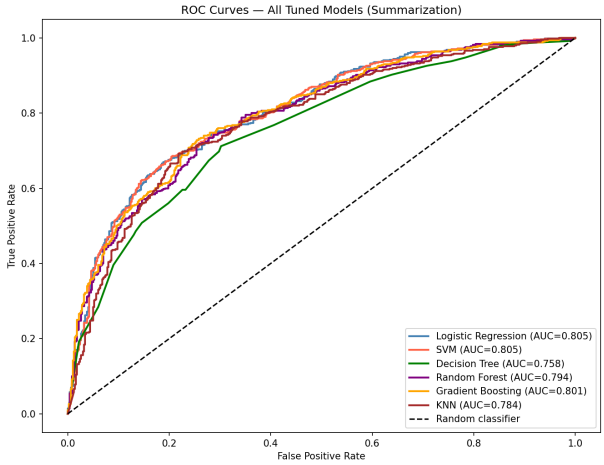


Figure 2: ROC curves for all six tuned models on the held-out test set. Decision Tree is clearly distinguished from the ensemble and linear methods.

Among semantic features, BLEU and ROUGE-1 contribute meaningfully to Random Forest, while the NLI score and semantic similarity contribute modest but consistent signal across models. Hedge count contributes near-zero importance across all three models, confirming it is not a useful signal for this task.

ROC Curve Analysis

Figure 2 shows ROC curves for all six tuned models. Decision Tree is clearly separated from the remaining five models, with AUC of 0.758 vs. 0.784–0.806 for the others. The five competitive models cluster tightly in the upper-left region, with SVM and Logistic Regression exhibiting the best early recall meaning they identify a higher proportion of hallucinations at low false positive rates, which is the practically desirable operating point for a hallucination detector.

Precision-Recall Tradeoff

A notable pattern across models is the inverse relationship between precision and recall. SVM and Logistic Regression achieve the highest precision (0.759 and 0.752) but the lowest recall (0.688 and 0.696), behaving as conservative detectors that rarely raise false alarms. Random Forest and Gradient Boosting achieve more balanced precision-recall tradeoffs (precision 0.738–0.746, recall 0.710–0.714),

Table 5: Tuned model results on held-out test set (1,000 rows).

Model	Accuracy	F1	Precision	Recall	AUC
Gradient Boosting	**0.734**	**0.728**	0.746	0.710	0.801
Random Forest	0.730	0.726	0.738	0.714	0.794
Logistic Regression	0.733	0.723	0.752	0.696	0.805
SVM	0.735	0.722	0.759	0.688	**0.806**
KNN	0.722	0.721	0.723	0.720	0.784
Decision Tree	0.698	0.691	0.708	0.674	0.758

Table 6: Tuned vs. untuned F1 comparison.

Model	Baseline F1	Tuned F1	ΔF1
Decision Tree	0.6662	0.6906	+0.024
KNN	0.7054	0.7214	+0.016
SVM	0.7331	0.7219	−0.011
Random Forest	0.7379	0.7256	−0.012
Gradient Boosting	0.7329	0.7275	−0.005
Logistic Regression	0.7381	0.7227	−0.015

Table 7: Comparison to existing hallucination detection approaches.

Approach	F1	Cost	Interpretable
LLM-as-a-Judge	∼0.85+	High	No
SelfCheckGPT	∼0.44	Medium	No
This work	**0.728**	**Low**	**Yes**

making them more suitable for deployment scenarios where missing a hallucination carries higher cost than a false alarm.

Comparison to Existing Work

On the HaluEval summarization subset, GPT-4 achieves ∼75% accuracy as a zero-shot hallucination recognizer (Zhang et al. 2023). The best model reported here (Gradient Boosting, F1=0.728) achieves comparable performance while requiring no frontier model API, no GPU, and no proprietary access at inference time. SelfCheckGPT achieves F1 ≈ 0.44 on adversarial QA benchmarks (Manakul, Liusie, and Gales 2023), significantly below both GPT-4 and the proposed approach. Table 7 summarizes this comparison.

Feature Distribution Analysis

Figure 3 shows the distribution of all eight features by label. The `response_len` plot confirms the residual length difference between classes. The `sem_sim` and `bertscore_f1` distributions overlap heavily, consistent with their low feature importance. The `nli_score` plot exhibits a bimodal pattern with both labels spiking at zero (neutral predictions) but with hallucinated summaries showing a larger spike at -1.0 (contradiction detected) and correct summaries showing a larger spike at $+1.0$ (entailment detected), confirming that NLI captures genuine but sparse signal.

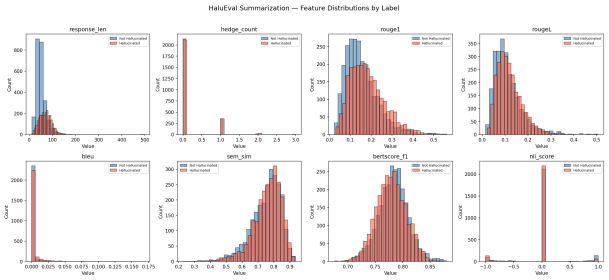


Figure 3: Feature distributions by label on HaluEval summarization. Blue: correct summaries. Red: hallucinated summaries.

Real-World and Social Impact

Potential Applications

Automated hallucination detection has immediate practical applications across several domains. In **healthcare**, LLMs are increasingly used to summarize patient records, generate clinical notes, and answer medical queries. A hallucination in a medication recommendation or diagnostic summary could directly endanger patient safety. A fast, interpretable detector of the kind described in this work could serve as a real-time filter before LLM outputs reach clinical staff.

In **legal and financial services**, hallucinated citations, incorrect statutes, or fabricated financial figures in LLM-generated documents carry serious liability risks. A lightweight CPU-deployable detector is particularly valuable in these regulated industries where explainability is legally required and computational budgets are constrained.

In **education**, students increasingly rely on LLMs for research assistance. An embedded hallucination detector in educational platforms could flag potentially incorrect content before it is accepted as fact, supporting critical thinking rather than undermining it.

Fairness and Ethical Considerations

Several fairness and ethical concerns arise in this work.

Dataset bias: The HaluEval summarization subset is derived from CNN/Daily Mail news articles, which reflect the editorial biases of Western English-language media. A detector trained on this data may not generalize to other domains, languages, or cultural contexts. Hallucination patterns in medical, legal, or scientific text may differ substantially from those in news summarization.

Disparate impact: If a hallucination detector is deployed in a content moderation or quality assurance pipeline, systematic false positives on certain topics or writing styles could lead to unfair suppression of content. The precision-recall tradeoff described in this work has direct fairness implications: high-precision detectors minimize false accusations of hallucination, while high-recall detectors minimize missed hallucinations.

Misuse potential: A hallucination detector could be repurposed to selectively flag or suppress content based on stylistic or topical features rather than factual accuracy, particularly if deployed without interpretability safeguards. The feature importance analysis provided in this work is a step toward transparency, but additional auditing is recommended before production deployment.

AI disclosure: This project was completed with assistance from Claude for brainstorming, debugging, and writing guidance. All code, analysis, and written content were reviewed, understood, and validated by the author, in accordance with course academic integrity policies.

Conclusion and Future Work

Summary of Findings

This paper presented a feature-based classification approach to hallucination detection in LLM outputs, with three principal findings.

First, dataset quality matters more than model choice. The initial use of the HaluEval QA subset produced inflated F1 scores of $\sim$0.95, driven almost entirely by response length rather than semantic hallucination signals. This length bias was identified through ablation analysis and feature importance inspection, and corrected by adopting the summarization subset. On this more rigorous benchmark, all six classifiers converged to a narrow F1 range of 0.691–0.728, confirming that the feature set is the bottleneck, not the model.

Second, hallucination detection is genuinely hard. An F1 of 0.728 on subtle summarization hallucinations is realistic and consistent with GPT-4's reported $\sim$75% accuracy on the same benchmark. Hallucinated summaries are designed to be plausible, borrowing vocabulary and structure from the source document while distorting factual content making surface-level features such as ROUGE and semantic similarity unreliable in isolation.

Third, domain shift is a fundamental challenge. When the trained classifier was applied to LLM responses on TruthfulQA, near-universal hallucination predictions were produced, reflecting a distribution mismatch between news summary features and conversational QA response features. This finding motivates domain-specific hallucination detectors trained on data matched to the deployment context.

Lessons Learned

The most important practical lesson from this work is that benchmark performance should always be interrogated before being trusted. A 96% F1 score on a hallucination detection task should prompt investigation, not celebration. The process of discovering, quantifying, and correcting the QA length bias is documented across the project notebooks as an example of responsible empirical practice.

A secondary lesson is that interpretability is not in tension with performance. The feature-based approach described here achieves results comparable to GPT-4 on the same benchmark while providing clear, auditable explanations for every prediction—a property that black-box approaches cannot offer.

Future Work

Several directions are identified for future investigation. **Domain-adaptive detectors** trained on data matched to specific deployment contexts (medical QA, legal summarization, conversational assistants) would address the domain shift limitation identified in this work. **Fine-tuned transformer classifiers** such as DeBERTa or RoBERTa, trained end-to-end on hallucination labels, would likely improve performance beyond the 0.728 F1 ceiling observed with classical feature-based approaches. **Token-level uncertainty features** such as output probability entropy, available from some inference APIs, represent a promising complement to the current feature set. Finally, extending the evaluation to additional LLMs including GPT-4o and Claude Sonnet would enable a more comprehensive cross-model analysis than was possible under free API tier constraints.

Acknowledgments

The author thanks the course staff of EECE 5644 at Northeastern University for feedback on the project proposal. API access was provided through free tiers of Mistral AI, Groq, and HuggingFace. Datasets were sourced from the HuggingFace Hub.

References

Breiman, L. 2001. Random Forests. *Machine Learning*, 45(1): 5–32.

Chen, T.; and Guestrin, C. 2016. XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.

Cortes, C.; and Vapnik, V. 1995. Support-Vector Networks. *Machine Learning*, 20(3): 273–297.

He, P.; Liu, X.; Gao, J.; and Chen, W. 2021. DeBERTa: Decoding-enhanced BERT with Disentangled Attention. *International Conference on Learning Representations*.

Hermann, K. M.; Kočiský, T.; Grefenstette, E.; Espeholt, L.; Kay, W.; Suleyman, M.; and Blunsom, P. 2015. Teaching Machines to Read and Comprehend. In *Advances in Neural Information Processing Systems*, 1693–1701.

Lin, C.-Y. 2004. ROUGE: A Package for Automatic Evaluation of Summaries. In *Text Summarization Branches Out*, 74–81.

Lin, S.; Hilton, J.; and Evans, O. 2022. TruthfulQA: Measuring How Models Mimic Human Falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 3214–3252.

Manakul, P.; Liusie, A.; and Gales, M. J. 2023. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 9004–9017.

Min, S.; Krishna, K.; Lyu, X.; Lewis, M.; Yih, W.-t.; Koh, P. W.; Iyyer, M.; Zettlemoyer, L.; and Hajishirzi, H. 2023. FActScoring: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 12076–12100.

Papineni, K.; Roukos, S.; Ward, T.; and Zhu, W.-J. 2002. BLEU: A Method for Automatic Evaluation of Machine Translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, 311–318.

Reimers, N.; and Gurevych, I. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*.

Zhang, T.; Kishore, V.; Wu, F.; Weinberger, K. Q.; and Artzi, Y. 2020. BERTScore: Evaluating Text Generation with BERT. In *International Conference on Learning Representations*.

Zhang, Y.; Li, Y.; Cui, L.; Cai, D.; Liu, L.; Fu, T.; Huang, X.; Zhao, E.; Zhang, Y.; Chen, Y.; Wang, L.; Luu, A. T.; Bi, W.; Shi, F.; and Shi, S. 2023. HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 6449–6464.